Science Olympiad Build Proposal: Robot Tour

Nathan Zhao[1]

[1] *The Pembroke Hill School, Kansas City, USA*

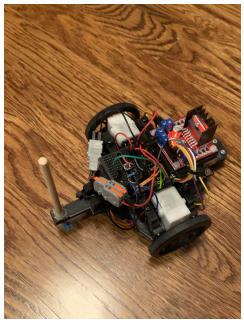
October 10, 2025

Abstract

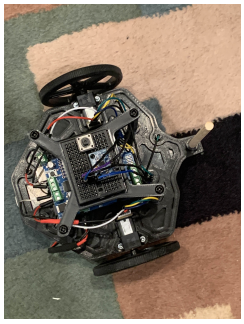
In this work, we present the design, implementation, and evaluation of a differential-drive robotic vehicle developed for the Science Olympiad Robot Tour event. The Robot Tour challenge requires a robot to autonomously traverse a prescribed grid-style track, enter specified gate zones, push water bottle payloads, and stop accurately at a target point, all while arriving as close as possible to a predetermined target time. Key performance metrics include positional accuracy (distance from target point), adherence to the target time, and successful traversal of required course features.

1 Previous Work

In past years, the challenge of robot tour has been purely movement and navigation; emphasis was placed upon having solid, consistent motion hardware, and relatively simple command queue-based code. Thus, devices from previous years have followed a straightforward formula for design. Previous iterations from 2024 (Figure 1a) and 2025 (Figure 1b) are depicted below.



(a) Device from 2024



(b) Device from 2025

Figure 1: Previous Iterations of Robot Tour Devices

For their torque output, ease of control, and relatively low costs, motions systems were based around brushed DC gearmotors, specifically the Pololu Romi plastic gearmotors in 2024, and generic N20 motors in 2025. To run motors with closed loop distance and speed control, selected motors included hall effect quadrature encoders on the motor shaft. Motor gearbox ratios were selected based on estimated required torque as well as encoder increments (ticks) per wheel revolution, which were calculated as $PPR * GR_{TOTAL}$. H-bridge motor drivers were used, specifically the L289N dual motor drivers in 2024, and a TB6612 Based I2C Shield in 2025. Both robots had a singular ball caster in the rear and two Pololu 60mm x 8 wheels, which provided decent traction and were simple to attach to the motors' 3mm d-shafts. Robot chassis were designed in CAD and FDM 3D-printed in PLA.

For ease of programming and wide availability, previous devices all used the ATMega328P microcontroller in the form of an Arduino UNO R3. Using the Arduino IDE, these could be quickly and easily reprogrammed in competition over USB connection with a laptop. Code was written in Arduino C++, and included basic closed-loop PID control algorithms for speed/distance, as well as a simple command queue system to input movements (e.g. `moveForward(1000mm)` or `turnRight()`).

1.1 Challenges and Limitations

However, these design choices introduced major limitations on performance. While performance in practice was often within 5 millimeter distance scores, these tests were done on smooth, clean surfaces. In competition, these scores usually jumped to the 4-6 centimeter range, with time scores or bonus zones having to be sacrificed for distance accuracy. Contrary to what one might expect from the rules, competition floors are rarely clean and often even bumpy, which has had severe impacts on previous devices' performances (see: 2024 and 2025 state competitions). With this in mind, as well as problems with microcontroller performance such as encoder misreads and slow control loop times, distance error quickly compounded and time control was exceedingly difficult. Without any orientation/heading feedback from an external inertial measurement unit (IMU), all movements had to be inferred from kinematics calculations based on wheel rotation, and thus the robot could not detect nor adjust for unexpected errors. An MPU6050 6-axis gyroscope was briefly experimented with in 2025 (sensor is shown attached to breadboard), but gyro drift made readings unusable.

Many of these limitations stem from issues with the motion hardware itself, such as robot traction (insufficient wheel contact area), center of mass (COM) instability, chassis rigidity, and lack of motor torque output. Issues with precision arise from the lack of feedback inputs (motor encoder data, no IMU, no other sensors), as well as the performance of the microcontroller processing them (low 16MHz clock speed, 8-bit only, slow interrupts, etc.). On the programming side, while the simple command motions do work, pauses in between moves and stationary turning are not optimal for speed control and can increase the impacts of track surface variance.

2 Proposed Invitational Device

Taking into consideration these limitations, as well as the additional challenge of moving a water bottle payload for the 2026 season, we propose the following hardware improvements for the first iteration of the device this year:

1. **Chassis Material**
   To improve chassis rigidity, robot parts will be printed in fiber-reinforced filaments (likely PLA-CF or PETG-CF) at high infill, instead of just PLA. Aluminum standoffs with Loctite will be used instead of 3D-printed/Nylon standoffs, which were known to loosen and introduce vibrations over time. Wheels and motors will also be double-shear supported with flanged ball bearings. Additionally, the design includes mounting for ball casters with integrated bearings, though empirical evidence suggests that any performance improvements from this will be trivial.

2. **Banebots T61 Wheels**
   To improve robot traction, Banebots T61 wheels will be used instead of Pololu 60mm x 8mm. Optimal durometer remains to be seen (based on final robot weight), but it will likely be either 30A or 20A Shore Hardness (Purple or Green). Custom wheel hubs will be used to interface with ball bearings, and include set screws for mounting (as opposed to just press-fit).

3. **Stepper Motors**
   To increase robot torque, Nema 17 42x42x23mm Bipolar Stepper Motors will be used instead of the previously used brushed DC gearmotors. Compared to the nominal 7.5 oz-in (5.3 N-cm) stall torque at a current of 0.67A, the stepper motors have a holding torque of 24.07 oz-in (17 N-cm) at 1.0A. While this change does increase power draw, it should still be within the limitations of the 8xAA NiMH 2800mAh Batteries.

   The use of stepper motors requires a change in motor driver hardware. We propose the use of TMC2208 stepper drivers. While the TMC2209 drivers offer higher performance and efficiency, the TMC2208 drivers are simpler to wire (and the author just happens to have a bunch of them lying around in his basement). With a base resolution of 200 steps per revolution, the microstepping capabilities of the drivers (up to 1/16), we can calculate the final resolution as $200 * \frac{1}{R}$ (for example, with a microstepping resolution of $R = \frac{1}{8}$, we get $200 * 8 = 1600$ microsteps per 1 full revolution of the motor shaft). This theoretically allows for higher resolution than offered by DC motor shaft encoders (approximately 1000 CPR), which were prone to miscounts.

4. **AS5600 Magnetic Encoders**
   It is a known issue that stepper motors may misstep when facing motor loads, which classifies stepper motor control as open-loop since they do not necessarily follow the commands sent to them by the driver. Thus, the design will include mounting for AS5600 magnetic encoders for closed-loop stepper control. These encoders give 12bit resolution angle measurement (4096 PPR) using a diametrically magnetized neodymium disc, and transmit information over I2C.

5. **RP2040 MCU**
   Compared to the ATMega328P, the RP2040 offers increased processing capabilities with higher power efficiency. The RP2040 is a dual-core 32-bit ARM Cortex-M0+ with a 133MHz clock speed, as opposed to the single-core 8-bit ATMega328P with a 16MHz clock speed. The additional SRAM (264KB vs. 2KB) and flash memory (2MB vs. 32KB) also allow for increased onboard algorithmic control features, such as Kalman filtering for IMU data, S-curve motion profiles, and continuous path smoothing (which has many benefits in this year's challenge).

   The RP2040 development board used is the Adafruit KB2040, which has sufficient pinout for all required connections, including two hardware I2C buses to interface with the two AS5600 encoders, which do not have configurable addresses.

6. **MPU9250/6500 IMU**
   Integrating heading and orientation data can help correct for drift, floor deviations, or disturbances. The MPU9250/6500 is a 9-DOF (Degree of Freedom) IMU that integrates a gyroscope, accelerometer, and magnetometer, and can be added on as either an I2C or SPI peripheral.

2.1 Differential Drive Kinematics

Let:

v_L	: linear velocity of left wheel
v_R	: linear velocity of right wheel
r	: wheel radius
L	: wheelbase (distance between wheels)
ω_L, ω_R	: angular velocities of left/right wheels
v, ω	: linear and angular velocity of robot
(x, y, θ)	: pose of the robot in world coordinates

Then:

$$v_L = rw_L, \qquad v_R = rw_R \tag{1}$$

The robot's forward linear and angular velocities are:

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{L} \tag{2}$$

Pose kinematics (non-holonomic constraint):

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \frac{v_R - v_L}{L} \tag{3}$$

For odometry integration, we update pose numerically (discrete-time integration):

$$x_{k+1} = x_k + v_k cos\theta_k \Delta t, \quad y_{k+1} = y_k + v_k sin\theta_k \Delta t \quad \theta_{k+1} = \theta_k + \omega_k \Delta t \tag{4}$$

Each wheel uses a velocity PID controller:

$$u_i(t) = K_P e_i(t) + K_I \int_0^t e_i(\tau)\, d\tau + K_D \frac{de_i(t)}{dt} \tag{5}$$

where $e_i(t) = v_{i,target}(t) - v_{i,measured}(t)$ and $i \in \{L, R\}$. The control outputs, u_L, u_R, modulates step frequency for each stepper driver.

To achieve a desired (v, w):

$$v_R = v + \frac{L}{2}\omega, \qquad v_L = v - \frac{L}{2}\omega \tag{6}$$